

ServiceDesk - Dokumentation

Projekt

Eingereicht von:

Lukas Ulovecs

Patrick Speiser

Raphael Rumpler

am

Fachhochschul-Bachelorstudiengang

Informatik

Virtualisierung und Versionierung

Wiener Neustadt, 15. Januar 2026

Inhaltsverzeichnis

1	Einleitung	4
2	Projektübersicht	5
2.1	Versionierung	5
2.2	Strategie	5
2.3	Container-Architektur	6
3	Build- und CI/CD-Pipeline	7
3.1	Security Scan	7
3.2	Build	7
3.3	Test	8
3.4	Docker Image Build & Push	8
4	Deployment der Anwendung	9

Abbildungsverzeichnis

1.1	ServiceDesk Dashboard	4
2.1	MSSQL-Container für die Datenbank	6
3.1	Erfolgreicher Lauf der CI/CD-Pipeline	8

1 Einleitung

Das vorliegende Dokument dient als Beschreibung des Gruppenprojekt **ServiceDesk** aus der Lehrveranstaltung Virtualisierung und Versionierung aus dem Bachelorstudiengang Informatik an der Fachhochschule Wiener Neustadt. Die entwickelte Software ist eine containerisierte Webanwendung, bestehend aus einem API-Service und einem Web-Frontend, wie es in Abbildung 2.1 gezeigt wird. Das Projekt spiegelt dabei die wesentlichen Inhalte und Konezpte der Lehrveranstaltung, wie Versionierung, Containerisierung durch Docker sowie Continuous Integration mittels Github Actions und Continuous Deployment (CI/CD) wieder.

Der Fokus liegt auf:

- klarer Projektstruktur
- automatisierter Secret-Detection
- automatisierten Builds sowie anschließende Tests
- sicherem Container-Deployment

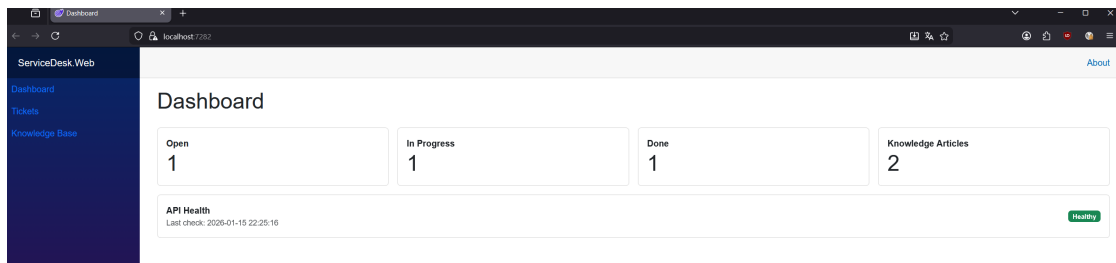


Abbildung 1.1: ServiceDesk Dashboard

2 Projektübersicht

Das Projekt umfasst mehrere klar getrennte Komponenten mit definierter Verantwortlichkeitsverteilung:

- **ServiceDesk.ApiService** – REST API (ASP.NET Core Web API) mit Geschäftslogik
- **ServiceDesk.Web** – Client-seitiges Web-Frontend (Blazor Server)
- **ServiceDesk.Data** – Datenpersistenzschicht (Entity Framework Core)
- **ServiceDesk.Contracts** – Gemeinsame Datenübertragungsobjekte (DTOs) und Modelle

2.1 Versionierung

Das Projekt wird über GitHub versioniert, da es einfache Zusammenarbeit sowie mittels GitHub Actions automatisierte Builds und Tests ermöglicht, sobald ein Commit in den Hauptbranch (main) erfolgt.

2.2 Strategie

Die Anwendung kann vollständig containerisiert ausgeliefert werden. Docker wird dabei verwendet, um die Anwendung unabhängig von der Zielumgebung (Windows/Linux/macOS) reproduzierbar bereitzustellen. Durch die Containerisierung wird sichergestellt, dass die Anwendung in Entwicklung, Test und Produktion unter identischen Bedingungen ausgeführt wird, was Deployments vereinfacht und Umgebungsprobleme reduziert.

- **API-Service** und **Web-Frontend** → Dockerfile pro Projekt → GitHub Actions Build & Push
- **Datenbank** → Verwalteter oder Docker-Container SQL Server

Für diese Projekt erfolgt die Bereitstellung der Datenbank sowohl in der Entwicklung mittels Aspire und Docker, als auch in der Produktion über einen Sql-Server in einem Container wie Abbildung 2.1 zeigt.

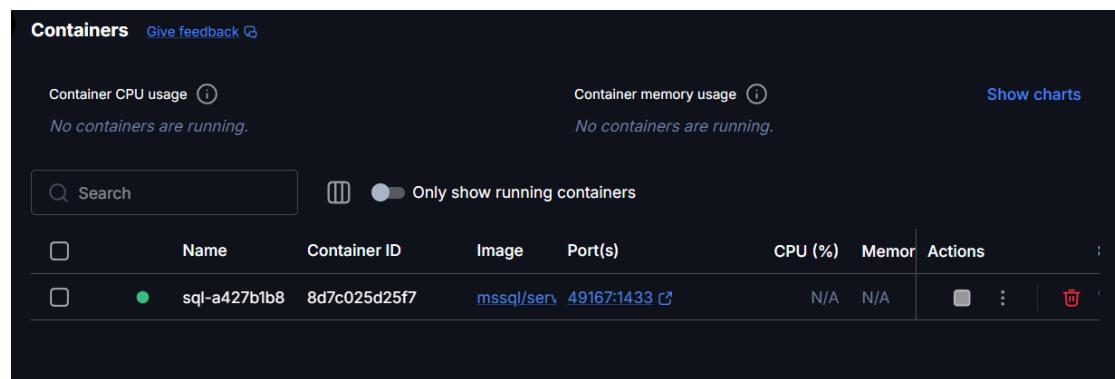


Abbildung 2.1: MSSQL-Container für die Datenbank

2.3 Container-Architektur

Die eigentliche Anwendung in der Produktion besteht aus drei Hauptcontainern, welche über ein entsprechendes docker-compose-file folgendermaßen definiert werden kann:

- **servicedesk-apiservice** – API Container (Port 8080)
- **servicedesk-web** – Web Frontend Container (Port 80)
- **servicedesk-db** – SQL Server (Port 1433)

In der Entwicklung verwendet Aspire Docker und Service Discovery zum Bereitstellen der Datenbank, wobei somit in Entwicklug, als auch in der Produktion die Datenbankschemata über Entity Framework Core Migrations (über `Database.MigrateAsync()`) verwaltet werden. Es ist also kein weiteres SetUp-File oder ähnliches für die Datenbank notwendig.

3 Build- und CI/CD-Pipeline

Zur Sicherstellung einer stets funktionierenden Software in entsprechender Qualität und sofortiger Mitteilung bei Abweichung oder kritischen Fehlern wurde eine CI/CD auf Github (`.github/workflows/dotnet-build.yml`) aufgesetzt. Die Pipeline wird automatisch bei jedem Push sowie bei jedem Pull Request auf den `main`-Branch ausgeführt und besteht aus folgenden vier Stages, welche nacheinander durchlaufen werden:

3.1 Security Scan

Im ersten Stage erfolgt eine sicherheitsrelevante Analyse des Quellcodes. Diese Phase dient dazu, potenzielle Sicherheitsrisiken (Secrets wie `ConnectionString` im Repo) bereits vor dem eigentlichen Build zu identifizieren und abzuberechnen.

- Secret Detection (Gitleaks): Überprüft das Repository auf versehentlich eingetragene Zugangsdaten wie Passwörter, API-Keys oder Tokens.
- Static Application Security Testing (Semgrep): Führt eine statische Codeanalyse durch, um bekannte Sicherheitsprobleme (wie SQL-Injection) oder unsichere Codepatterns sowie Konfigurationsfehler frühzeitig zu erkennen.

Durch diese Maßnahmen wird verhindert, dass sicherheitskritischer Code in spätere Pipeline-Stages oder in den Produktivbetrieb gelangen.

3.2 Build

Nach erfolgreichem Abschluss der Sicherheitsprüfungen beginnt die dotnet Build-Phase, in der die gesamte Anwendung kompiliert wird:

- Wiederherstellung aller .NET-Abhängigkeiten (`dotnet restore`)
- Kompilierung der gesamten Solution im Release-Modus
- Erzeugung von Build-Artefakten zur Wiederverwendung in nachfolgenden Pipeline-Stages (Test-Stage)

3.3 Test

In der Test-Stage werden automatisierte Tests auf Basis der zuvor erstellten Build-Artefakte ausgeführt, um die funktionale Korrektheit der Anwendung sicherzustellen.

- Ausführung aller vorhandenen Unit-Tests
- Validierung der Geschäftslogik und zentraler Komponenten

Fehlschlagende Tests führen zu einem sofortigen Abbruch der Pipeline, wodurch verhindert wird, dass fehlerhafter Code weiterverarbeitet oder veröffentlicht wird.

3.4 Docker Image Build & Push

Im letzten Stage erfolgt die Containerisierung der Anwendung. Hierbei werden für den ApiServer (backend) als auch für das Web-Frontend separate Docker Images erstellt und jeweils an eine Docker-Registry gepusht.

- Verwendung eines Multi-Stage Docker Builds zur Reduktion der Image-Größe
- Erstellung eines separaten Images für den API-Service und das Web-Frontend
- Den Tag `latest` anhängen
- Push der Images in eine Container-Registry

Die daraus entstandenen Images können anschließend von der angegebenen Registry gepullt und über ein Docker-Compose-Setup bereitgestellt werden.

Abbildung 3.1 zeigt einen erfolgreichen Lauf der CI/CD-Pipeline mit allen vier Stages.

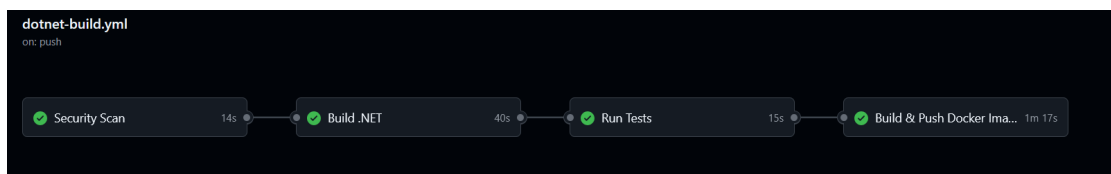


Abbildung 3.1: Erfolgreicher Lauf der CI/CD-Pipeline

4 Deployment der Anwendung

Das Deployment der Anwendung erfolgt in zwei Schritten:

1. Build der Docker-Images in der CI/CD-Pipeline (siehe vorheriges Kapitel)
2. Automatisiertes Deployment mithilfe von Docker Compose über Dokploy

Zunächst wurde mit dem Befehl `aspire publish` ein `docker-compose.yml`-File für das gesamte Projekt generiert. Anschließend wurden für die relevanten Projektkomponenten – insbesondere API und Web-Frontend – eigene `Dockerfile`-Definitionen erstellt, die den Build-Prozess der Images beschreiben.

Die erzeugten Images werden bei jedem Push innerhalb der CI/CD-Pipeline automatisch gebaut und in die Docker Registry (Docker Hub) hochgeladen. Jeder Push in das Repository löst anschließend ein automatisiertes Deployment auf Dokploy aus. Dokploy verwendet dabei das zuvor generierte `docker-compose.yml`-File zusammen mit den aus der Registry geladenen Images, um die vollständige Anwendung als Docker-Compose-Stack bereitzustellen.

Die deployte Applikation wird schließlich über einen Traefik Reverse Proxy öffentlich zugänglich gemacht. Optional kann eine benutzerdefinierte Domain konfiguriert und der Zugriff mittels HTTPS abgesichert werden.